Vidyavardhini's College of Engineering & Technology

Department of Computer Science and Engineering (Data Science)

EXPERIMENT ASSESSMENT

ACADEMIC YEAR 2025-26

Course: Machine Learning Lab

Course code: CSL604

Year: TE SEM: VI

Experiment No. 1
AIM:- To study and implement basic Machine Learning Libraries like Scipy, Scikit Learn, Tensor Flow and Keras.
Name: Satyam Maurya
Roll Number: 62
Date of Performance:05/01/2026
Date of Submission:12/01/2026

Evaluation

Performance Indicator	Max. Marks	Marks Obtained
Performance	5	
Understanding	5	
Journal work and timely submission.	10	
Total	**20**	

Performance Indicator	Exceed Expectations (EE)	Meet Expectations (ME)	Below Expectations (BE)
Performance	5	3	2
Understanding	5	3	2
Journal work and timely submission.	10	8	4

Checked by

Name of Faculty : Ms. Nivedha Raut

Signature :

Date :

Aim - To study and implement basic Machine Learning libraries like Scipy, Scikit Learn, Tensor Flow and Keras.

Objective - To introduce platforms such as Anaconda and COLAB suitable to machine learning.

Requirement -
1. Google Colab
2. VS code
3. Anaconda
4. Jupyter Notebook

Theory –
Python is an interpreted, object-oriented, high-level programming language with dynamic semantics. Its high-level built in data structures, combined with dynamic typing and dynamic binding, make it very attractive for Rapid Application Development, as well as for use as a scripting or glue language to connect existing components together. Python's simple, easy to learn syntax emphasizes readability and therefore reduces the cost of program maintenance. Python supports modules and packages, which encourages program modularity and code reuse. The Python interpreter and the extensive standard library are available in source or binary form without charge for all major platforms and can be freely distributed.

Often, programmers fall in love with Python because of the increased productivity it provides. Since there is no compilation step, the edit-test-debug cycle is incredibly fast. Debugging Python programs is easy: a bug or bad input will never cause a segmentation fault. Instead, when the interpreter discovers an error, it raises an exception. When the program doesn't catch the exception, the interpreter prints a stack trace. A source level debugger allows inspection of local and global variables, evaluation of arbitrary expressions, setting breakpoints, stepping through the code a line at a time, and so on. The debugger is written in Python itself, testifying to Python's introspective power. On the other hand, often the quickest way to debug a program is to add a few print statements to the source: the fast edit-test-debug cycle makes this simple approach very effective.

Observations:

Q1. Write steps to install Jupyter notebook. Explain Google Colab.

Ans:Make sure Python is installed.

1. Open **Command Prompt / Terminal**.
2. Type:`pip install notebook`
4. After installation, start Jupyter by typing:

```
jupyter notebook
```

5. Browser will open → start coding.

Google Colab Explanation:

Google Colab (Colaboratory) is an **online platform provided by Google** to run Python and Machine Learning programs **without installing anything**.

Key Features

- Runs in your browser
- Free to use
- Provides free GPU/TPU for machine learning
- Supports Jupyter Notebook (.ipynb files)
- Easy to share like Google Docs

Q2. What are python modules? Name some commonly used built-in modules in Python.

Ans: Python Modules are files that contain Python code (functions, classes, variables) that can be reused in other Python programs. Instead of writing code again and again, you can simply import the module and use its features. Modules help in organizing code, improving readability, and promoting reusability.

Commonly Used Built-in Python Modules:

- **math** – for mathematical functions
- **random** – for generating random numbers
- **datetime** – for working with dates and time
- **os** – for interacting with the operating system
- **sys** – for system-related operations
- **json** – for handling JSON data
- **statistics** – for statistical calculations
- **re** – for regular expressions (pattern matching)

Code:
#scipy

```
from scipy import stats
[redacted]
data=[10,20,30,40,50]
mean= stats.tmean(data)
std_dev=stats.tstd(data)
[redacted]
print("mean:", mean)
print(std_dev)
```

Output:

```
mean: 30.0
15.811388300841896
```

#tensorflow

```
import tensorflow as tf

X = [1,2,3,4]
y = [2,4,6,8]

model = tf.keras.Sequential([
    tf.keras.layers.Dense(10, input_shape=[1]),
    tf.keras.layers.Dense(1)
])

model.compile(optimizer='sgd', loss='mse')
model.fit(X, y, epochs=50, verbose=0)

print("Prediction for 5 =", model.predict([5]))
```

Output:

```
Prediction for 5 = [[9.452632]]
```

#scikit

```
from sklearn.linear_model import LinearRegression
import numpy as np

X = np.array([[1],[2],[3],[4]])    # Features
y = np.array([2,4,6,8])            # Labels

model = LinearRegression()
model.fit(X, y)

print("Prediction for 5 =", model.predict([[5]]))
```

#keras

```
import numpy as np
import tensorflow as tf

# Training data
X = np.array([1, 2, 3, 4], dtype=float)
y = np.array([2, 4, 6, 8], dtype=float)

# Build model
model = tf.keras.Sequential([
```

```
    tf.keras.Input(shape=(1,)),
    tf.keras.layers.Dense(10, activation="relu"),
    tf.keras.layers.Dense(1)
])

model.compile(optimizer='sgd', loss='mse')

# Train model
model.fit(X, y, epochs=50, verbose=0)

# Test
print("Prediction for 5 =", model.predict(np.array([5], dtype=float)))
```

output:

```
Prediction for 5 = [[9.781549]]
```

Conclusion:

The study and implementation of SciPy, Scikit-Learn, TensorFlow, and Keras provide a strong foundation for understanding Machine Learning concepts. SciPy helps in performing scientific and statistical calculations easily. Scikit-Learn offers powerful tools for building basic machine learning models. TensorFlow allows the creation of deep learning models with high performance and flexibility. Keras simplifies neural network development, making it easier for beginners to work with AI. Overall, these libraries make Machine Learning efficient, accessible, and practical for real-world applications.